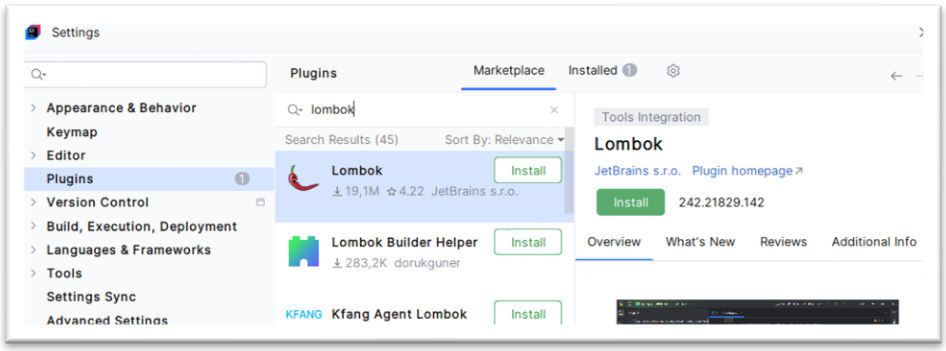
Spring Data JPA

Minta projekt

1. Induló projekt elkészítése „spring initializr” segítségével
   A spring initializr (sic), spring boot projekt template-et szolgáltat előre konfigurálva.
   a. - IntelliJ IDEA Ultimate esetén használhatjuk az integrált projekt létrehozót
      - Egyéb esetben látogassuk meg a https://start.spring.io/ oldalt
   b. Inicializáljunk egy projektet az alábbiak szerint:
      i. Maven projektkezelő
      ii. Java nyelv
      iii. Legfrissebb verziójú Spring Boot
      iv. Maven koordináták:
         Group ID: `hu.unideb.inf`
         Artifact ID: `SpringDataFX`
         Name: `SpringDataFX`
         Description: `Tetszőleges szöveg`
      v. Dependencies:
         `Spring Web`
         `Spring Data JPA`
         `Lombok`
         `H2 Database`
   c. Generáljuk le a sablont és (webes esetben kicsomagolás után) töltsük be a fejlesztői környezetbe.
2. Hozzunk létre egy Person POJO-t Lombok segítségével
   a. Ellenőrizzük, hogy a Lombok plugin engedélyezve van-e a fejlesztői környezetben. (Settings -> Plugins -> „Keressünk a Lombok szóra”



   b. Egy model nevű alcsomagban hozzunk létre egy Person osztályt `id`, `name` és `dateOfBirth` attribútumokkal.
   c. Jelöljük az osztályt `@Data` lombok annotációval, majd figyeljük meg ennek hatását.
   d. Használjuk az `@AllArgsConstructor` és `@NoArgsConstructor` annotációkat is és figyeljük meg a hatásukat.
   e. Végül használjuk a `@Builder` lombok annotációt és példányosítsunk egy Person objektumot ennek használatával.
3. JPA segítségével képezzük le a Person osztályt entitásként egy H2 adattáblába

a. Használjuk az `@Entitiy`, `@Id` és `@GeneratedValue` annotációkat a megfelelő helyen.
b. Állítsuk be a H2 Console elérhetőségét az `application.properties` fájlban

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

A vastaggal szedett rész, ami fontos, de a többi is állítható opcionálisan.
c. Készítsünk egy `PersonRepository` nevű interfészt, a `JpaRepository<Person, Long>` gyermekeként. Opcionálisan használhatjuk a `CrudRepository`-t, ami a `JpaRepository` szülője, illetve egyes esetekben hasznos lehet az interfész előtt a `@Repository` annotáció használata.
d. Adjunk a `SpringDataFxApplication` osztályhoz egy `PersonRepository` referencia változót.
e. Terjesszük ki a `CommandLineRunner` interfészt, hogy egy rövid példa kódot futtathassunk.
f. Az implementálandó run(...){} metódusban hozzunk létre egy Person példányt, majd mentsük el az alábbi sorral:

```
personRepository.save(p);
```

g. Honnan kap a `personRepository` példányt? Onnan, hogy használjuk a definíciójában az `@Autowired` annotációt.
h. Futtassuk a kódot és ellenőrizzük a H2 DBMS PERSON táblájának tartalmát.

Minta projekt kiterjesztése JavaFX felülettel

1. Adjuk hozzá a pom.xml fájlunkhoz a megfelelő függőséget és plugint:
   https://openjfx.io/openjfx-docs//#maven
   https://mvnrepository.com/artifact/org.openjfx/javafx-fxml
2. Hozzunk létre a projektünkön belül egy új alcsomagot `frontend` néven, azon belül pedig egy osztályt `JavaFXApplication` néven
   a. A `JavaFXApplication` terjessze ki a `javafx Application` absztrakt osztályát
   b. Ennek megfelelően implementálni kell a start() metódust

```
Parent root = FXMLLoader.load(
          getClass().getResource("/fxml/MainPage.fxml"));
stage.setScene(new Scene(root));
stage.show();
```

   c. A `resources` mappa alatt hozzunk létre egy új mappát `fxml` néven. Ebbe generáljuk a `MainPage.fxml` fájlt.
   d. A MainPage.fxml-ben szereplő kontroller elérési útja legyen
   hu.unideb.inf.SpringDataFX.frontend.controller.MainPageController
   (A kékkel jelölt rész elérhet, ha nem pontosan ezt a project nevet adtuk meg.)
3. Indítsuk el a SpringBoot alkalmazást a JavaFX alkalmazásból.
   a. A `JavaFXApplication` `start(…)` metódusának első soraként futtassuk a Spring alkalmazásunkat a `SpringApplication.run(…)` metódussal. A parameter a Spring alkalmazásunk osztály (.class végződéssel).
   b. Figyeljük meg, mi történik, ha az alkalmazást többször egymás után próbáljuk futtatni.

c. Az `application.properties` fájlban állítsuk át az alapértelmezett portot `server.port=8081` formában, majd a kód újra futtatható egy alkalommal.
d. A SpringApplication.run(...) metódus visszatérési értékét mentsük ki egy példányváltozóba (`ConfigurableApplicationContext ctx;`).
e. A `JavaFXApplication` osztályban írjuk felül a `stop(…)` metődust úgy, hogy az bezárja a Spring alkalmazást a `ctx`-en keresztül (`ctx.close();`).
f. Futtassuk többször a JavaFX alkalmazást és figyeljük meg, hogy mikor zárul be a hívott Spring alkalmazás.

Kiegészítő feladatok

1. Szüntessük meg a JavaFXApplication és a SpringFXApplication közötti erős csatoltságot.
2. Próbáljuk ki, milyen további funciókat ad számunkra a JPARepository interfész
3. Olvassuk ki egy személy adatait a neve alapján
   https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html